

Modul 1 — Persiapan Tool Development dan Konfigurasi Lingkungan

Pertemuan	Minggu 1 dari 14
Fase	Fase 1 — Fondasi teknis
Durasi	210 menit efektif di kelas (3 jam 30 menit); tugas mandiri di luar kelas hingga 120 menit (2 jam)
Ketertelusuran	Batasan umum §1.3.4; antarmuka perangkat lunak §3.1.3; keputusan arsitektur Fase 0

Output utama: Repository Git `kantin-multi-tenant` dengan commit awal yang bersih.

Tujuan Pembelajaran

- ☐ Memverifikasi versi tool dan ekstensi PHP yang diwajibkan Laravel serta driver aplikasi. (C2 – Memahami)
- ☐ Membuat proyek Laravel 13 dengan starter kit Livewire 4 dan autentikasi dasar. (C3 – Menerapkan)
- ☐ Menghubungkan Laravel ke MariaDB dan Redis melalui variabel lingkungan. (C3 – Menerapkan)
- ☐ Menjalankan server, Vite, queue worker, migrasi, test, dan formatter sebagai health check. (C4 – Menganalisis)

Output Project

- ☐ Repository Git `kantin-multi-tenant` dengan commit awal yang bersih.
- ☐ Laravel 13 + Livewire 4 berjalan pada `http://localhost:8000`.
- ☐ MariaDB dan Redis sehat; `.env` lokal tidak masuk Git.
- ☐ Bukti `php artisan test`, `npm run build`, dan Pint berhasil.

Pendahuluan

Pada mata kuliah pemrograman web dasar, Anda sudah terbiasa menjalankan PHP secara native dan menghubungkannya langsung ke database. Pertemuan pertama ini menaikkan level itu: Anda membangun lingkungan pengembangan Laravel yang dapat direplikasi oleh komputer lain, bukan sekadar berjalan di komputer Anda sendiri. Sebelum kode domain (fitur kantin multi-tenant) ditulis, Anda memeriksa PHP 8.3+, Composer (pengelola dependency/pustaka PHP), Node/NPM (runtime dan pengelola paket JavaScript untuk membangun aset frontend), Git, MariaDB, Redis, dan editor kode; kemudian membuat proyek Laravel 13 dengan starter kit Livewire. Laravel 13 mensyaratkan minimal PHP 8.3 [S1].

MariaDB menyimpan data transaksional, sedangkan Redis dipakai untuk session, cache, cart, dan queue (antrian pekerjaan yang diproses di latar belakang, terpisah dari request pengguna). Pemisahan ini penting karena data transaksi membutuhkan constraint (aturan integritas data pada level database) dan ACID (jaminan transaksi yang atomik, konsisten, terisolasi, dan tahan gagal), sementara state sementara membutuhkan akses cepat. Hasil akhir minggu ini adalah proyek yang dapat dijalankan, dimigrasikan, diuji, dan dikompilasi oleh mesin pengembang lain dengan konfigurasi yang sama — kriteria yang akan Anda buktikan sendiri lewat checklist di akhir modul.

Pengetahuan Teknis dan Penjelasan Mandiri (Laravel 13)

FOKUS TEKNIS • toolchain yang dapat direplikasi, batas konfigurasi, dan proses lokal yang wajib hidup sebelum fitur domain ditulis.

- ☐ **Runtime dan toolchain.** Runtime adalah program yang benar-benar mengeksekusi kode Anda saat aplikasi berjalan; toolchain adalah kumpulan alat pendukung yang bekerja sama membangun dan menjalankan proyek. Laravel 13 memerlukan PHP 8.3 atau lebih baru. Siapkan PHP beserta ekstensi yang diminta,

Composer (pengelola dependency PHP), Laravel installer, Git, serta Node/NPM atau Bun untuk aset frontend.

- ❑ **Pembuatan proyek.** Perintah `laravel new` memandu pemilihan starter kit (paket awal berisi struktur proyek, autentikasi, dan tampilan siap pakai); pilih Livewire agar fondasi autentikasi dan UI selaras dengan arsitektur modul. `composer run dev` menjalankan server lokal, worker queue (proses latar belakang yang mengerjakan pekerjaan asinkron), dan Vite (alat pembangun aset frontend yang mengolah CSS dan JavaScript) dalam satu proses pengembangan.
- ❑ **Batas konfigurasi.** `.env` adalah berkas nilai konfigurasi khusus mesin lokal Anda (host database, password, token), sedangkan berkas di `config/` menerjemahkannya menjadi konfigurasi aplikasi yang bernama dan dapat dipanggil dari kode. Simpan nilai per lingkungan di `.env`, hanya commit placeholder pada `.env.example`, panggil `env()` hanya dari berkas konfigurasi, lalu akses nilainya dengan `config()` pada kode aplikasi. Setelah `config:cache` (perintah yang menggabungkan seluruh konfigurasi menjadi satu cache), `.env` tidak lagi dimuat saat request.
- ❑ **Layanan data.** Gunakan koneksi MariaDB untuk data transaksional dan Redis untuk cache, session, cart, serta queue. Pisahkan database/prefix pengujian agar `migrate:fresh` dan flush Redis tidak menyentuh data kerja. Bila layanan lokal dikemas sebagai container, versikan `compose.yaml`.
- ❑ **Zona waktu dan mode produksi.** Simpan waktu transaksi dalam UTC, lakukan konversi ke `Asia/Jakarta` pada batas presentasi, dan pastikan `APP_DEBUG=false` di produksi agar detail konfigurasi tidak bocor.

TUJUAN PEMAHAMAN · Bagian ini menjelaskan hubungan antaralat, bukan sekadar daftar instalasi. Setelah membacanya, mahasiswa harus dapat menerangkan proses yang berjalan, asal setiap konfigurasi, dan cara membedakan gangguan aplikasi dari gangguan infrastruktur lokal. Rujukan tetap dicantumkan untuk traceability, bukan sebagai prasyarat memahami materi.

Alur kerja runtime dan toolchain. PHP mengeksekusi kode backend Laravel, Composer memasang package PHP dan membentuk autoloader, sedangkan Node/npm menjalankan Vite untuk mengolah CSS serta JavaScript. Git menyimpan perubahan sumber, bukan hasil build sementara atau secret. Saat `composer run dev` dijalankan, beberapa proses dapat hidup bersamaan: HTTP server menerima request, queue worker mengerjakan job asinkron, dan Vite menyajikan aset dengan hot reload. Karena proses itu berbeda, halaman web dapat terbuka walaupun aset gagal dibangun atau job tidak pernah diproses. Diagnosis selalu dimulai dengan mengidentifikasi proses mana yang gagal, lalu membaca log proses tersebut.

Dari `.env` menuju `config()`. File `.env` berisi nilai yang berubah menurut lingkungan, misalnya host database, password Redis, atau token gateway. File di dalam `config/` menerjemahkan nilai tersebut menjadi konfigurasi aplikasi yang bernama. Kode domain tidak boleh memanggil `env()` secara langsung; kode membaca `config('services.qris.token')` agar tetap bekerja setelah `config:cache`. Pada produksi, Laravel menggabungkan konfigurasi menjadi cache sehingga perubahan `.env` baru berlaku setelah cache dibangun ulang dan proses panjang di-reload. `.env.example` hanya memuat nama variabel dan contoh aman, sedangkan `.env` nyata tidak masuk Git.

Peran MariaDB dan Redis. MariaDB adalah sumber kebenaran untuk data yang harus tahan kegagalan, seperti tenant, order, payment, stok, dan ledger. Redis menyimpan data berumur pendek atau data koordinasi berkecepatan tinggi, seperti session, cart, cache, lock, dan antrian job. Data penting tidak boleh hanya berada di Redis karena TTL (batas waktu hidup data sebelum otomatis terhapus), eviction (penghapusan paksa saat memori penuh), atau flush (pengosongan manual) dapat menghilangkannya. Sebaliknya, cart sementara tidak perlu membebani tabel transaksi. Gunakan database, prefix Redis, dan credential berbeda untuk development serta testing agar `migrate:fresh`, flush cache, atau parallel test tidak merusak lingkungan lain.

Pemeriksaan kesiapan lingkungan. Lingkungan dianggap siap bila versi PHP memenuhi Laravel 13, Composer dapat menyelesaikan dependency, npm dapat membangun aset, koneksi MariaDB dan Redis berhasil, migrasi selesai, serta test bawaan lulus. Kesalahan `connection refused` biasanya menunjukkan service atau port belum aktif; `access denied` menunjukkan credential; class tidak ditemukan mengarah ke autoload/dependency;

sedangkan halaman tanpa gaya mengarah ke Vite. Simpan hasil pemeriksaan versi dan perintah bootstrap dalam README agar seluruh anggota tim mereproduksi lingkungan yang sama [L13-INS] [L13-TEST].

CONTOH MANDIRI – config/services.php dan service pemakai • PHP

```
// config/services.php
return [
    'qris' => [
        'base_url' => env('QRIS_BASE_URL'),
        'token' => env('QRIS_TOKEN'),
    ],
];

// Di dalam adapter: baca repository config, bukan env().
$baseUrl = config('services.qris.base_url');
$token = config('services.qris.token');
```

LANDASAN TEKNIK – terminal pada root workspace • BASH

```
php -v
composer global require laravel/installer
laravel new kantin-multi-tenant
cd kantin-multi-tenant
npm install && npm run build
php artisan migrate
composer run dev
```

Jembatan dari Mata Kuliah Prasyarat

HUBUNGAN · Dari instalasi PHP native menuju environment Laravel yang dapat direplikasi, dipisahkan dari secret, dan mempunyai quality gate.

Sebelum Coding: Pause & Predict

- Mengapa database development dan testing harus berbeda?
- Mengapa .env tidak boleh masuk Git?
- Proses apa saja yang harus hidup saat mengembangkan realtime?

Tulis jawaban sementara maksimal tiga kalimat per pertanyaan. Revisi jawaban setelah fase Observe dan tandai perubahan pemahaman.

Prasyarat

- ☐ Akses terminal dengan izin memasang software atau Docker Desktop/Engine.
- ☐ Git identity sudah dikonfigurasi; port 8000, 8080, 3306, dan 6379 tersedia.
- ☐ Salinan SRS v2, mockup revisi, ERD, dan SQL baseline Fase 0.

Rencana Waktu — 3 Jam 30 Menit Efektif di Kelas

Tahap	Menit	Aktivitas	Checkpoint
1	30	Orientasi artefak dan target repository	Anda dapat menjelaskan fungsi MariaDB, Redis, Reverb, Livewire, dan queue dalam arsitektur.
2	30	Instal dan audit toolchain	`check-environment.sh` memberi status OK untuk tool utama atau catatan eksplisit untuk client yang diganti Docker.
3	30	Buat proyek Laravel 13	`php artisan --version` menampilkan Laravel Framework 13.x dan halaman login dapat dirender.
4	30	Konfigurasi MariaDB, Redis, locale, dan waktu	`php artisan db:show` terhubung dan `redis-cli -p 6379 ping` mengembalikan PONG.
5	30	Instal Livewire/Reverb dan validasi proses dev	HTTP, Vite, queue worker, dan Reverb dapat berjalan tanpa error konfigurasi.
6	30	Quality gate dan baseline test	Semua command exit code 0; tidak ada secret pada `git status` atau `git diff --cached`.

Tahap	Menit	Aktivitas	Checkpoint
7	30	Dokumentasi dan commit pertemuan	Clone bersih dapat dijalankan mengikuti README; working tree bersih setelah commit.

Total efektif di kelas: 210 menit (7 tahap × 30 menit). Tugas mandiri di luar kelas: hingga 120 menit (2 jam) untuk pekerjaan sejenis yang bersifat mengulang — misalnya bila di kelas membuat CRUD untuk 2 tabel, di luar kelas kerjakan CRUD untuk 3 tabel lainnya.

Tahap demi Tahap

Tahap 1 — Orientasi artefak dan target repository (30 menit)

Sampaikan — Apa dan mengapa

Mari kita baca batasan teknologi pada SRS dan tentukan satu root project. Pisahkan artefak desain dari source aplikasi agar perubahan requirement tetap dapat ditelusuri. Bayangkan alurnya seperti dapur sebelum jam buka: alat, bahan, dan area kerja perlu siap sebelum pesanan pertama masuk. Langkah ini penting karena target repository yang jelas mencegah artefak desain tercampur dengan source code, sehingga jejak perubahan requirement tetap bisa ditelusuri.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Mari kita buat folder kerja tanpa spasi yang menyulitkan CLI.
- ☐ Mari kita salin SRS, mockup, ERD, dan SQL ke folder `docs/` setelah proyek dibuat.
- ☐ Kita akan tetapkan branch awal `main` dan pola commit `type(scope): message`.

Amati — Hasil yang kita harapkan

Anda dapat menjelaskan fungsi MariaDB, Redis, Reverb, Livewire, dan queue dalam arsitektur. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Perhatikan pemisahan artefak desain dan source code: SRS, mockup, ERD, serta SQL baseline masuk ke `docs/`, sedangkan kode aplikasi tumbuh di root project — pemisahan ini yang membuat perubahan requirement tetap mudah ditelusuri kembali ke sumbernya. Bagus, fondasi tahap ini sudah terbentuk.

Tahap 2 — Instal dan audit toolchain (30 menit)

Sampaikan — Apa dan mengapa

Mari kita jalankan skrip pemeriksa. Jika satu item gagal, perbaiki PATH atau instal ekstensi sebelum melanjutkan; jangan menumpuk masalah environment ke tahap coding. Langkah ini penting karena toolchain yang belum diverifikasi akan memunculkan error membingungkan di tahap-tahap berikutnya, padahal penyebabnya sekadar ekstensi PHP yang belum aktif.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Kita akan periksa `php -v`, `composer --version`, `node -v`, `npm -v`, dan `git --version`.
- ☐ Kita akan periksa `pdo_mysql`, `mbstring`, `openssl`, `ctype`, `curl`, `fileinfo`, `xml`, dan `tokenizer`.
- ☐ Mari kita pilih layanan native atau jalankan `docker compose up -d`.

Amati — Hasil yang kita harapkan

`check-environment.sh` memberi status OK untuk tool utama atau catatan eksplisit untuk client yang diganti Docker. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Perhatikan bahwa script pemeriksa hanya melaporkan gejala di permukaan (tool ada atau tidak); penyebab sebenarnya sering berada di PATH sistem operasi atau ekstensi PHP yang belum diaktifkan. Mantap, kita sudah menutup satu risiko penting.

Pro Tip untuk Artisan

Di VS Code, aktifkan PHP Intelephense dan gunakan integrated terminal agar file, perintah, serta error dapat kita lihat dalam satu layar.

Tahap 3 — Buat proyek Laravel 13 (30 menit)**Sampaikan — Apa dan mengapa**

Mari kita gunakan Laravel installer agar struktur proyek dan script `composer run dev` mengikuti dokumentasi resmi [S2]. Pada prompt, pilih starter kit Livewire; untuk kerangka test, pilih PHPUnit agar konsisten dengan proyek acuan kelas (bukan Pest — keduanya sama-sama didukung installer resmi, tetapi modul-modul berikutnya memakai PHPUnit). Langkah ini penting karena starter kit menentukan fondasi autentikasi dan struktur UI yang akan dipakai di seluruh modul berikutnya.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Mari kita jalankan `composer create-project --prefer-dist laravel/laravel kantin-multi-tenant` lalu pilih Livewire starter kit.
- ☐ Kita akan masuk ke folder proyek, jalankan `npm install` dan `npm run build`.

Amati — Hasil yang kita harapkan

`php artisan --version` menampilkan Laravel Framework 13.x dan halaman login dapat dirender. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Perhatikan bahwa starter kit hanya menyiapkan fondasi; struktur route, controller, dan Livewire component domain kantin multi-tenant baru akan dibangun pada modul-modul berikutnya. Kerja bagus, hasilnya kini dapat kita uji dengan lebih percaya diri.

Tahap 4 — Konfigurasi MariaDB, Redis, locale, dan waktu (30 menit)**Sampaikan — Apa dan mengapa**

Mari kita salin `.env.example` menjadi `.env`, buat APP_KEY, lalu isi koneksi. Gunakan koneksi Laravel `mysql` untuk MariaDB. Zona waktu aplikasi dan database disimpan dalam UTC dengan presentasi ke Asia/Jakarta (WIB). Langkah ini penting karena kesalahan konfigurasi koneksi database dan Redis adalah sumber error paling umum pada pertemuan pertama.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Mari kita jalankan `php artisan key:generate`.
- ☐ Mari kita isi `DB_*`, `REDIS_*`, `SESSION_DRIVER`, `CACHE_STORE`, dan `QUEUE_CONNECTION` dalam `.env`.
- ☐ Mari kita buat database uji terpisah; jangan memakai database pengembangan untuk test otomatis.
- ☐ Mari kita jalankan `php artisan migrate:fresh --seed` pada database kosong.

Amati — Hasil yang kita harapkan

`php artisan db:show` terhubung dan `redis-cli -p 6379 ping` mengembalikan PONG. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Redis menyimpan intent sementara dengan cepat, tetapi harga dan stok tetap dibaca ulang dari MariaDB. Karena itu cart boleh berubah, sedangkan order yang committed harus historis. Sip, alur ini sekarang lebih mudah ditelusuri.

Tahap 5 — Instal Livewire/Reverb dan validasi proses dev (30 menit)**Sampaikan — Apa dan mengapa**

Jika starter kit belum memasang Livewire, gunakan Composer; instalasi Livewire memakai package auto-discovery (mekanisme yang mendaftarkan otomatis service provider paket ke aplikasi) [S3]. Reverb (server WebSocket resmi Laravel untuk fitur realtime) dipasang melalui perintah broadcasting [S4]. Langkah ini penting karena Livewire dan Reverb adalah fondasi interaksi realtime yang akan dipakai berulang kali pada modul-modul berikutnya.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Mari kita pastikan `composer require livewire/livewire:^4.0` tercatat.
- ☐ Mari kita jalankan `php artisan install:broadcasting` dan isi variabel Reverb lokal.
- ☐ Mari kita jalankan `composer run dev`; pada terminal terpisah uji `php artisan reverb:start` jika belum disertakan script.
- ☐ Kita akan daftar, login, logout, lalu pantau log browser dan Laravel.

Amati — Hasil yang kita harapkan

HTTP, Vite, queue worker, dan Reverb dapat berjalan tanpa error konfigurasi. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Event realtime adalah sinyal perubahan, bukan sumber kebenaran. Klien tetap harus dapat memuat snapshot terbaru dari database ketika pertama kali membuka layar atau setelah reconnect. Hebat, increment kita mulai terlihat seperti fitur yang utuh.

Tahap 6 — Quality gate dan baseline test (30 menit)**Sampaikan — Apa dan mengapa**

Quality gate (pemeriksaan otomatis yang wajib lulus sebelum kode dianggap selesai) sejak minggu pertama mencegah tumpukan utang teknis di kemudian hari. Laravel menyediakan PHPUnit sebagai kerangka test dan perintah `php artisan test` [S5]. Langkah ini penting karena quality gate yang berjalan sejak pertemuan pertama mencegah utang teknis menumpuk di modul-modul berikutnya.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Mari kita jalankan `php artisan test`.
- ☐ Mari kita jalankan `./vendor/bin/pint --test`; format dengan Pint bila perlu.
- ☐ Mari kita jalankan `npm run build` dan pastikan tidak ada asset error.
- ☐ Mari kita tambahkan `.editorconfig`, aturan line ending, dan abaikan `.env`.

Amati — Hasil yang kita harapkan

Semua command exit code 0; tidak ada secret pada `git status` atau `git diff --cached`. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Test adalah requirement yang dapat dieksekusi. Feature test memeriksa perilaku, integration test memeriksa kontrak antarkomponen, dan E2E membuktikan alur pengguna dari awal sampai akhir. Hampir selesai—quality gate akan membantu kita memastikan semuanya tetap sehat.

Tahap 7 — Dokumentasi dan commit pertemuan (30 menit)**Sampaikan — Apa dan mengapa**

Kita akan tuliskan cara bootstrap agar komputer kedua dapat mereproduksi hasil tanpa percakapan tambahan. Langkah ini penting karena dokumentasi setup yang jelas memungkinkan siapa pun mereproduksi lingkungan kerja tanpa bertanya langsung kepada Anda.

Lakukan — Mari kita kerjakan

Ikuti perubahan kecil berikut satu per satu. Simpan file setelah setiap kelompok perubahan agar penyebab error tetap mudah ditemukan.

- ☐ Mari kita buat `README.md` berisi requirements, setup, run, test, dan troubleshooting.
- ☐ Mari kita simpan tangkapan layar halaman login dan output test di jurnal praktikum, bukan repository bila berukuran besar.
- ☐ Kita akan commit `chore(setup): bootstrap Laravel 13 development environment`.

Amati — Hasil yang kita harapkan

Clone bersih dapat dijalankan mengikuti README; working tree bersih setelah commit. Jika hasilnya belum sama, jangan khawatir—cocokkan pesan error dengan bagian troubleshooting sebelum melanjutkan.

Jelaskan — Apa yang terjadi di balik layar

Commit kecil membuat perubahan mudah direview dan dibatalkan. README menjelaskan cara menjalankan sistem, sedangkan ADR menyimpan alasan di balik keputusan arsitektur. Selamat! Increment minggu ini sudah siap dibuktikan dan di-commit.

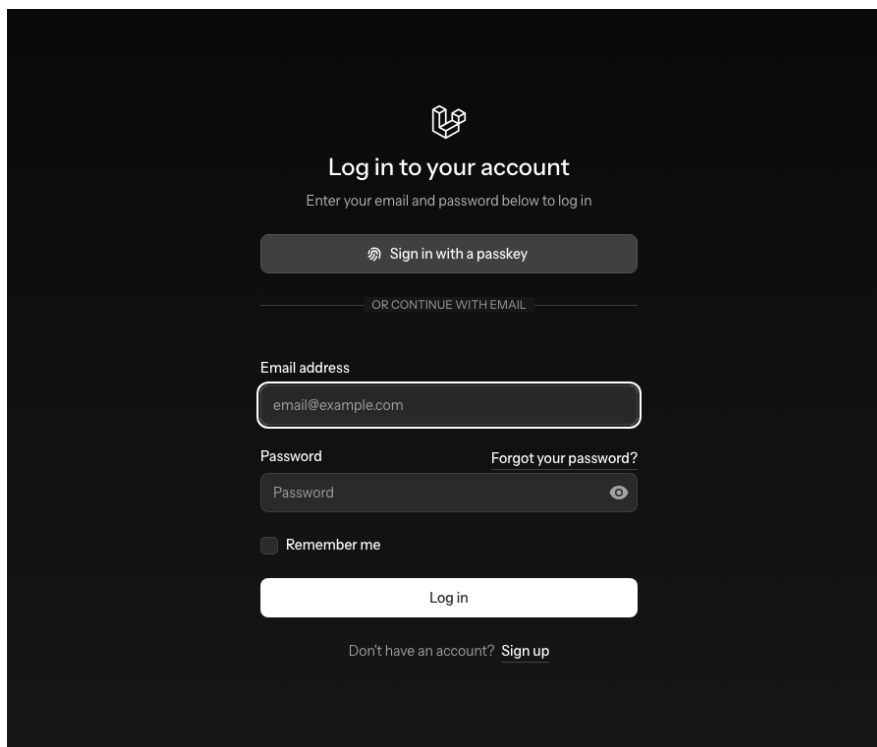
Guided Lab: Predict → Try → Observe → Explain

TRY · Perintah berikut dikelompokkan per tema (pemeriksaan versi, pembuatan proyek, konfigurasi), bukan sebagai satu urutan tunggal dari atas ke bawah — jalankan tiap kelompok sesuai tahap yang relevan, satu kelompok pada satu waktu. Berhenti saat muncul error yang belum dipahami; gunakan tabel troubleshooting sebelum melanjutkan.

```
Lokasi: terminal pada root repository kantin-multi-tenant • MODUL 1 – TERMINAL • BASH
php artisan test
npm run build
php -v
composer --version
npm -v
git --version
docker compose up -d
composer run dev
composer global require laravel/installer
laravel new kantin-multi-tenant
npm install
php artisan --version
php artisan key:generate
php artisan migrate:fresh --seed
```

Output Nyata Terverifikasi

Bukti eksekusi nyata Pertemuan 1 pada lingkungan pengembangan (bukan ilustrasi).



Gambar M1. Halaman login (Laravel 13 + starter kit Livewire) berjalan di `http://localhost:8000`.

```
$ php artisan --version
Laravel Framework 13.25.0
$ php artisan test
Tests: 33 passed (81 assertions)
```

Observe: Cocokkan dengan Checkpoint

- ☐ Anda dapat menjelaskan fungsi MariaDB, Redis, Reverb, Livewire, dan queue dalam arsitektur.

- ☐ `check-environment.sh` memberi status OK untuk tool utama atau catatan eksplisit untuk client yang diganti Docker.
- ☐ `php artisan --version` menampilkan Laravel Framework 13.x dan halaman login dapat dirender.
- ☐ `php artisan db:show` terhubung dan `redis-cli -p 6379 ping` mengembalikan PONG.
- ☐ HTTP, Vite, queue worker, dan Reverb dapat berjalan tanpa error konfigurasi.

Explain: Ucapkan dengan Istilah Sendiri

- Mengapa database development dan testing harus berbeda?
- Mengapa .env tidak boleh masuk Git?

Cuplikan Kode dan Penjelasan

File yang dibuat: 01_setup/.env.example dan 01_setup/compose.yaml

```
APP_NAME="Kantin Multi-Tenant"
APP_ENV=local
APP_URL=http://localhost:8000
APP_LOCALE=id
APP_TIMEZONE=UTC

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=kantin

SESSION_DRIVER=redis
CACHE_STORE=redis
QUEUE_CONNECTION=redis
BROADCAST_CONNECTION=reverb
```

- ☐ `DB\_CONNECTION=mysql` tetap benar untuk MariaDB karena Laravel memakai driver PDO MySQL.
- ☐ Session, cache, dan queue diberi Redis agar state sementara tidak menambah beban tabel transaksi.
- ☐ Nilai secret nyata hanya berada di `.env`; `.env.example` memakai placeholder aman dan boleh di-commit.

Cuplikan menonjolkan pola inti. Guided Lab memberi urutan pembuatan, verifikasi, dan bukti; lengkapi relasi, factory, namespace, serta binding sesuai increment yang telah dibangun.

Verifikasi dan Kriteria Selesai

- ☐ Framework 13.x dan PHP ≥8.3 terdeteksi.
- ☐ Registrasi/login berfungsi.
- ☐ Migrasi dan koneksi Redis berhasil.
- ☐ Test, build, dan formatter lulus.
- ☐ README setup dapat diikuti ulang.

Jika Macet: Troubleshooting Cepat

Tiga gejala berikut paling sering muncul pada pertemuan pertama. Cocokkan pesan error Anda dengan kolom Gejala, lalu ikuti perbaikannya sebelum melanjutkan ke tahap berikutnya.

Gejala	Penyebab mungkin	Perbaikan	Bukti pulih
could not find driver	pdo_mysql belum aktif	Aktifkan ekstensi lalu restart PHP	php -m memuat pdo_mysql
Port 3306 atau 6379 bentrok	Service lain masih aktif	Ganti port Compose dan .env	db:show dan redis ping berhasil
Vite manifest tidak ada	Asset belum diinstal atau dibangun	Jalankan npm install dan npm run build	Halaman memuat CSS dan JS

Tantangan Bertingkat

LEVEL INTI · Buat script pemeriksa toolchain dengan exit code nonzero saat gagal.

LEVEL LANJUT · Tambahkan healthcheck MariaDB dan Redis pada Compose.

Evidence Praktikum

- ☐ Repository Git `kantin-multi-tenant` dengan commit awal yang bersih.
- ☐ Laravel 13 + Livewire 4 berjalan pada `http://localhost:8000`.
- ☐ MariaDB dan Redis sehat; `.env` lokal tidak masuk Git.
- ☐ Bukti `php artisan test`, `npm run build`, dan Pint berhasil.

Exit Ticket — Jawab Sebelum Commit

1. Mengapa database development dan testing harus berbeda?

Jawaban:

2. Mengapa .env tidak boleh masuk Git?

Jawaban:

3. Proses apa saja yang harus hidup saat mengembangkan realtime?

Jawaban: